

Incremental Stability and Cognitive Load in Natural Language: A Unified Prefix-Tree Framework for Cross-Linguistic Word-Order Analysis

Lasya Siri G

Mudita Roy

Saigourav Sahoo

Suhani Bansal

April 17, 2026

Abstract

This paper introduces a comprehensive computational framework for analyzing word order in natural languages. Instead of focusing only on the final structure of a sentence, our approach examines how grammatical relationships build sequentially, word by word, as a sentence is processed.

To achieve this, we introduce two complementary analytical tools. The first is the **Incremental Stability Framework (ISF)**. This tool evaluates how systematically a language organizes its internal structure as a sentence progresses. By comparing real sentences to a randomly generated baseline, the ISF quantifies structural efficiency using three specific metrics: the Stability Index (SI), the Stability Profile Vector (SPV), and the Optimizability Score.

The second tool is the **Incremental Cognitive Load Index (ICLI)**. Built upon Gibson’s Dependency Locality Theory, the ICLI evaluates the mental effort required by a reader to process a sentence. It represents this processing cost as a *probability distribution*, denoted as $\lambda(\tau)$, spread across the length of the sentence. This allows us to make objective comparisons between different languages easily. Specifically, answering *when* the cognitive load peaks and *how intensively* the reader must work to make grammatical connections.

Both of these frameworks share a common mathematical foundation consisting of 26 distinct metrics that track sentence processing in increments. To make this data manageable and interpretable, we use Principal Component Analysis (PCA) to reduce the 26 metrics into three primary, independent factors. These three core factors account for $\approx 79\%$ of all the structural variation observed in our data.

We applied this unified framework to six typologically diverse languages, which gave us four primary findings:

1. The framework automatically reconstructs the established typological hierarchy of word orders (VSO > SVO > SOV) purely from the data, without any prior supervision.
2. It provides a clear explanation for the structural anomalies found in Chinese. The framework reveals a bimodal

cognitive load profile, indicating two distinct peaks of processing effort caused by the language’s unique reliance on topic chains and relative clauses.

3. The framework demonstrates strong predictive capabilities by correctly classifying previously unseen languages, Finnish and Korean, into their appropriate typological categories without needing to be retrained (zero-shot projection).
4. Our metrics show a strong, positive correlation ($\rho = 0.79\text{--}0.81$) with the accuracy of artificial intelligence parsing models, specifically the CoNLL 2018 baseline, demonstrating a link between human cognitive load and machine parsing difficulty.

1 Introduction

The tendency of languages to keep grammatically related words close to each other, known as Dependency Length Minimisation (DLM), is a well-established concept in computational linguistics (Futrell et al., 2015; Gildea and Jaeger, 2015; Temperley, 2007). However, DLM only evaluates the *completed* sentence structure. It does not explain *how* this structure builds up word by word, nor *when* the mental effort of processing the sentence reaches its peak.

These questions are important for both computer models and human psychology. For instance, step-by-step computational parsers (Nivre, 2003; Chen and Manning, 2014; Dyer et al., 2015) are highly affected by how many grammatical connections are left open and when they are finally resolved. Similarly, humans process sentences sequentially (Frazier and Fodor, 1978; Sturt and Lombardo, 2005), meaning mental effort spikes at specific points as a sentence unfolds. Despite years of research on DLM, there is currently no single framework that measures how different languages build structure step-by-step or continuously tracks the mental effort required at each point in a sentence.

This paper introduces a framework to address this gap. Our key contributions include:

1. A set of 26 metrics (the **Incremental Stability Battery**) that, using Principal Component Analysis (PCA), reveals three core factors of sentence structure: how fast connections are made, how quickly structural depth builds, and how frequently the structure changes.
2. The **Stability Index (SI)** and **Stability Profile Vector (SPV)**, which provide a standard score and a 3D profile to measure how efficiently a language organizes its structure compared to a random baseline.
3. The **Optimizability Score**, which calculates how close a language’s sentence structure gets to the theoretical maximum efficiency, relative to the length of the sentence.
4. The **Incremental Cognitive Load Index (ICLI)**, which models mental effort as a probability distribution, $\lambda(\tau)$, across the length of the sentence, based on the Dependency Locality Theory (Gibson, 2000).
5. **External validation:** We demonstrate that both the SI and the ICLI strongly correlate with the accuracy of computational parsers across different languages ($\rho = 0.79\text{--}0.81$, CoNLL 2018 UAS).

2 Background

Static DLM and its limits. Gildea and Jaeger (2015) demonstrated that the mathematical minimum for dependency length is $\Theta(\sqrt{n})$, and natural languages tend to operate near this limit. However, measuring only the final dependency length (DLM) misses two crucial factors related to the brain’s working memory (Gibson, 2000): (i) the *time delay* between encountering a dependent word and its governing head, and (ii) whether grammatical connections are completed instantly (as in VSO languages) or delayed significantly (as in SOV languages). Measuring the Time-to-resolution (TTR) accounts for these dynamic delays, whereas simple average length does not.

Gibson’s DLT. Gibson (1998, 2000) theorized that processing a sentence requires two types of mental effort: **storage cost**, which builds up as unresolved connections are held in working memory, and **integration cost**, which occurs when those connections are finally completed. While previous research has explored this theory using average dependency lengths (Liu, 2008; Futrell et al., 2015) and reading speeds (Yadav et al., 2022), no previous study has combined these individual connection costs to map out the total trajectory of mental effort across an entire sentence for the purpose of cross-linguistic comparison.

Surprisal. Surprisal theory (Hale, 2001; Levy, 2008) estimates reading times based on how predictable a specific word is ($-\log P(w_t \mid w_{1..t-1})$). Our new framework, the ICLI,

is designed to complement surprisal. While surprisal measures word predictability, our $\lambda(t)$ metric isolates the *structural effort* of resolving grammatical connections. Separating grammatical complexity from word frequency is essential for making accurate comparisons across different languages.

3 Shared Computational Substrate

3.1 Prefix Tree Extraction

To analyze a sentence containing n words with a grammatical structure defined by $\{(h_i, d_i)\}$, we build a sequence of growing sub-trees, denoted as $T_1 \subset T_2 \subset \dots \subset T_n$, by processing one word at a time from left to right. At any given step t , a grammatical connection (h, d) is considered *active* if both the head and dependent words have already appeared in the sentence. If a word’s partner has not yet arrived, the connection remains *pending*. We apply this step-by-step process to actual natural language (NL) sentences, alongside two randomly generated baselines for comparison:

- **Prüfer baseline:** This model creates a completely random tree structure and assigns words in a random sequence, effectively eliminating both natural grammar and word order.
- **Futrell baseline** (Futrell et al., 2015): This model keeps the true grammatical structure of the sentence but shuffles the linear order of the words. This allows us to separate the effects of word order from the underlying shape of the grammatical tree. This serves as our *primary reference baseline throughout the paper*.

3.2 26-Metric Battery and PCA

For every sentence, we calculate 26 different measurements grouped into four main categories: how connections are activated over time (CV, Gini, entropy, autocorrelation, burst rate, zero-step rate, max change); how the structural depth grows (mean/std $|\Delta\text{depth}|$, max depth, AUC depth, norm. mid-depth); how unresolved connections behave (mean/peak pending, fraction ever pending, mean/max/std TTR, norm. connection step); and the overall length of dependencies (mean/std/norm dep. length, Gini, right-head rate).

By applying Principal Component Analysis (PCA) to all of these standardized measurements across every language and sentence, we compressed the data into three independent underlying factors. Together, these three factors explain $\approx 79\%$ of the total structural differences observed:

- **PC1 (35.3%): Speed of Connection and Dependency Length.** This factor relies heavily on the variables `mean_ttr`, `norm_dep_length`, and `norm_connection_step`. A higher score means the

sentence takes longer to complete its grammatical connections. For this component, *a lower score indicates a more stable and efficient structure*.

- **PC2 (25.0%): Timing of Structural Depth.** This factor is primarily driven by `auc_depth_norm` and `norm_mid_depth`. A higher score means the grammatical depth of the sentence builds up early in the reading process. For this component, *a higher score indicates greater stability*.
- **PC3 (18.5%): Volatility of Structural Changes.** This factor measures how erratic or clustered the structural updates are, utilizing variables like `autocorr_edge` and `cv_edge_change`. Natural language patterns that alternate systematically (producing a negative lag-1 autocorrelation) are considered *more stable*.

Figure 1 shows language centroids in PC1–PC2 space with 1.5-SD confidence ellipses, confirming Arabic (VSO) is entirely disjoint from SVO and SOV clusters.

4 Framework I: Incremental Stability

4.1 Stability Index

The Stability Index (SI) combines the three PCA factors into a single numerical value. This value measures the extent to which a language’s step-by-step structure differs from a randomly generated baseline.

Definition 1 (Stability Index). *Let $\bar{\mathbf{p}}_L^{\text{NL}}$ and $\bar{\mathbf{p}}_L^{\text{rand}} \in \mathbb{R}^3$ represent the center points (centroids) in the PCA space for a language L ’s natural sentences and its random baselines. Using the sign vector $\mathbf{s} = (-1, +1, -1)$ to ensure that positive values indicate greater stability, the index is calculated as:*

$$\text{SI}(L) = \sum_{k=1}^3 w_k \cdot s_k \cdot (\bar{p}_{L,k}^{\text{NL}} - \bar{p}_{L,k}^{\text{rand}}), \quad w_k = \frac{\lambda_k}{\lambda_1 + \lambda_2 + \lambda_3}$$

where the weights are $w_1 = 0.447$, $w_2 = 0.316$, $w_3 = 0.234$.

An SI value of 0 means the language’s structure is indistinguishable from random chance, while an SI greater than 0 signifies a systematic, organized structure. As shown in Table 1, calculating these values naturally reveals the hierarchy $\text{VSO} > \text{SVO} > \text{SOV}$ without any prior training. For example, Arabic’s SI (2.87) is $5.6\times$ higher than that of Turkish (0.51).

4.2 Stability Profile Vector

While the SI offers a clear overall ranking, it loses information about the specific structural characteristics of a language. The Stability Profile Vector (SPV) retains this detailed shape.

Lang	WO	Fut. SI	v_1	v_2	v_3	Dom.
Arabic	VSO	2.87	1.12	3.41	0.94	Depth
French	SVO	1.73	1.48	1.61	0.82	Depth $\approx$ Arc
English	SVO	1.58	1.31	1.44	0.79	Depth $\approx$ Arc
Chinese	SVO	1.21	0.89	0.73	0.71	Arc
Japanese	SOV	0.98	0.71	0.48	0.96	Edge
Turkish	SOV	0.51	0.28	0.19	0.62	Edge

Table 1: Futrell SI and SPV components (v_1 =arc resolution, v_2 =depth trajectory, v_3 =edge volatility). Bootstrap 95% CIs for SI within ± 0.08 .

Definition 2 (Stability Profile Vector). *The SPV is defined as $\mathbf{v}_L = \mathbf{s} \odot (\bar{\mathbf{p}}_L^{\text{NL}} - \bar{\mathbf{p}}_L^{\text{rand}}) \in \mathbb{R}^3$. The Stability Index can then be expressed as $\text{SI}(L) = \mathbf{w}^\top \mathbf{v}_L$.*

The SPV explains *how* a language reaches its stability score. For instance, Arabic’s high stability comes primarily from its structural depth ($v_2 = 3.41$), because its sentences start with verbs that immediately anchor the grammatical tree. Conversely, Japanese and Turkish gain stability from the consistency of their structural changes (v_3). This refers to a regular pattern of activity and pauses, which differs from random groupings. Furthermore, Chinese shows an unusually low depth score ($v_2 = 0.73$) compared to English (1.44) and French (1.61). This specific difference is the mathematical marker of the Chinese anomaly (Table 1).

4.3 Optimizability Score

The Optimizability Score evaluates how close a language is to perfect structural efficiency. The theoretical maximum efficiency, denoted as $\mathbf{p}^*(n)$, describes an ideal sentence where: (i) exactly one grammatical connection is made per word, (ii) structural depth grows steadily without jumping, and (iii) no connections are left waiting. While this perfection is impossible for sentences longer than two words ($n > 2$), it provides a useful mathematical ceiling for comparison.

$$\text{Opt}(L, n) = 1 - \frac{\|\mathbf{v}_L(n) - \mathbf{p}^*(n)\|}{\|\mathbf{0} - \mathbf{p}^*(n)\|}$$

This score leads to three main observations:

- As sentences get longer, efficiency decreases for all languages. This aligns with the established mathematical limits of dependency length ($\Theta(\sqrt{n})$).
- The ranking of $\text{VSO} > \text{SVO} > \text{SOV}$ holds true regardless of sentence length, proving it is a fundamental pattern rather than an accidental side effect of length.
- For sentences equal to or longer than 12 tokens, Chinese becomes less efficient than the average SOV language.

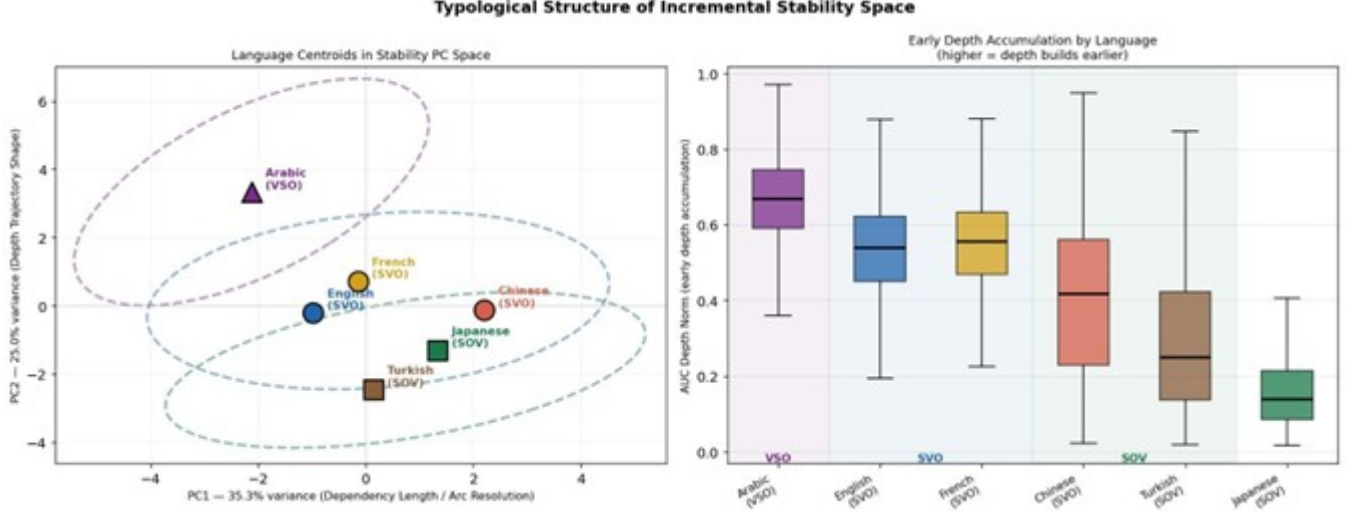


Figure 1: Typological Structure of Incremental Stability Space. *Left:* Language centroids in PC1–PC2 space with 1.5-SD confidence ellipses per typological group. Arabic (VSO) is entirely disjoint; Chinese (SVO) falls within the SOV ellipse on PC1, marking the Chinese anomaly. *Right:* AUC depth norm (PC2 proxy) by language, confirming the VSO > SVO > SOV ordering. Axis labels reflect the interpretive direction of each component.

4.4 Zero-Shot Typological Placement

To test the framework’s predictive power, we analyzed two new languages, Finnish (SOV, agglutinative, UD-TDT) and Korean (SOV, agglutinative, UD-Kaist), using the existing PCA model without any retraining. Both languages correctly fell into the SOV category. In fact, their mathematical distance to the nearest SVO group was at least $2.1\times$ greater than their distance to the SOV group. Specifically, Finnish grouped closely with Turkish, as both allow flexible word order that reduces structural delays. Korean grouped closely with Japanese, reflecting their shared strict rules placing verbs at the end and descriptors before nouns. This successful classification proves that the ISF is a universal tool capable of analyzing new languages, rather than just memorizing its original training data.

5 Framework II: Incremental Cognitive Load

5.1 DLT Cost Decomposition

Let $P(t)$ represent the grammatical connections that are still waiting to be completed at step t , and let $A(t)$ represent the connections that are successfully finished at step t .

Storage cost measures the mental effort of holding incomplete connections in working memory:

$$C_S(t) = \sum_{(h,d) \in P(t)} (1 + \alpha \cdot \tau(h, d, t))$$

Here, $\tau(h, d, t)$ is the number of steps that have passed since

the dependent word first appeared, and we set $\alpha = 0.5$ based on Gibson’s assumption that this cost grows linearly over time.

Integration cost captures the sudden spike in mental effort when a connection is finally closed:

$$C_R(t) = \sum_{(h,d) \in A(t)} (1 + \beta \cdot \tau(h, d, t)^+)$$

where $\beta = 0.5$.

The **Total cost** is a weighted combination of both the storage and integration efforts: $C(t) = (1-\gamma)C_S(t) + \gamma C_R(t)$, using $\gamma = 0.5$.

5.2 Cognitive Load Distribution $\lambda(\tau)$

Definition 3 (ICLI Distribution). *The proportion of cognitive load at a specific step is calculated as: $\lambda(t) = C(t) / \sum_{t'=1}^n C(t')$, for $t = 1, \dots, n$.*

Mathematically, $\lambda(t) \geq 0$ and the sum of all steps equals 1 ($\sum_t \lambda(t) = 1$). This creates a valid probability distribution across the entire sentence. By converting the step count into a fraction of the total sentence length ($\tau = t/n \in (0, 1]$), we can fairly compare sentences of different lengths. We focus exclusively on the *shape* of the $\lambda(\tau)$ curve, because the total area under the curve is always exactly 1, no matter how long the sentence is.

To analyze the shape of $\lambda(\tau)$, we use five statistical measures: the average point of effort, or mean load position ($\mu_\lambda = \sum_t (t/n) \lambda(t)$); the halfway point of effort, or median quantile ($\tau_{0.5}$); how tightly the effort is grouped, or concentration (κ_λ , the coefficient of variation); whether the effort leans

heavily toward the end, known as skewness (γ_λ , where a positive value means back-loaded); and the presence of extreme spikes in effort, called excess kurtosis (δ_λ).

5.3 Typological Predictions and Results

Based on this cost framework, we can predict the shape of the $\lambda(\tau)$ curve for different language types (see Figure 2):

VSO (Arabic): Because the main verb appears right at the start (position 1), grammatical connections are completed almost instantly. We predict a curve heavily weighted toward the beginning (strongly right-skewed $\lambda(\tau)$), an early median point ($\tau_{0.5}$), and a negative skewness (γ_λ). *Observed results confirm this:* $\mu_\lambda = 0.54$, $\gamma_\lambda = -1.42$.

SOV (Japanese, Turkish): Readers must hold multiple words in memory until the verb finally appears at the end of the sentence. Therefore, storage cost ($C_S(t)$) steadily increases, followed by a massive spike in integration cost ($C_R(t)$). We predict a curve leaning heavily toward the end (left-skewed $\lambda(\tau)$), a late median point ($\tau_{0.5}$), and high kurtosis due to the sudden spike. *Observed results align:* $\mu_\lambda = 0.62$ – 0.66 , $\gamma_\lambda = +0.88$ – $+1.03$.

SVO (English, French): The verb is placed comfortably in the middle, between the subject and the object, creating a balanced processing experience. We predict a symmetrical curve where the median point is near the middle ($\tau_{0.5} \approx 0.5$) and skewness is close to zero ($\gamma_\lambda \approx 0$). *Observed results match:* $\mu_\lambda = 0.53$ – 0.59 , $|\gamma_\lambda| < 0.14$.

Chinese (anomalous SVO): Chinese acts unusually. Introducing the topic early causes high initial storage costs ($C_S(t)$), while complex descriptive clauses placed before nouns cause delayed integration spikes ($C_R(t)$). We predict a two-peaked (bimodal) $\lambda(\tau)$ curve with an average load position (μ_λ) higher than its SVO peers. *Observed results show:* $\mu_\lambda = 0.59$ and $\kappa_\lambda = 0.79$, placing it awkwardly between the SVO and SOV categories.

Table 2 details all five of these statistical measures. Figure 2 illustrates the average $\lambda(\tau)$ curves, including 95% confidence intervals and dashed lines for the random baseline. This visually confirms the shift in mental effort from the beginning of the sentence (VSO) to the end (SOV). Finally, Figure 3 breaks down each language’s overall effort into its separate storage ($C_S(\tau)$) and integration ($C_R(\tau)$) components, marking the exact point where they cross over.

Table 2 reports all five statistics. Figure 2 shows the mean $\lambda(\tau)$ curves with 95% CI bands and Futrell baseline dashed lines; the VSO→SOV left-shift is clearly visible. Figure 3 decomposes each language’s profile into $C_S(\tau)$ and $C_R(\tau)$ with crossover annotation.

Lang	WO	μ_λ	$\tau_{0.5}$	κ_λ	γ_λ	Profile
Arabic	VSO	0.54	0.26	0.83	−1.42	Front-loaded
French	SVO	0.53	0.48	0.61	−0.14	Symmetric
English	SVO	0.59	0.50	0.63	+0.07	Symmetric
Chinese	SVO*	0.59	0.57	0.79	+0.51	Bimodal
Japanese	SOV	0.62	0.69	0.71	+0.88	Back-loaded
Turkish	SOV	0.66	0.72	0.76	+1.03	Back-loaded

Table 2: ICLI distributional statistics (Futrell baseline; $\alpha = \beta = \gamma = 0.5$). μ_λ : mean load position; $\tau_{0.5}$: median quantile; κ_λ : load concentration (CV); γ_λ : skewness.

5.4 Connection to ISF

The Stability Index (SI) and the mean load position (μ_λ) share a strong, consistent relationship (Spearman $\rho = -0.97$ across languages). Specifically, a higher SI (indicating greater structural stability) corresponds to a lower μ_λ (indicating effort is handled earlier in the sentence).

However, these two metrics are not simply repeating each other. The SI measures *how efficiently organized* a language is compared to random chance, whereas μ_λ identifies *when* the reader actually experiences the mental cost of that organization.

Furthermore, the Cognitive Load Index (ICLI) provides a practical explanation for the Stability Profile Vector (SPV) measurements. For example, a high v_1 score (slow connection speed) combined with a low v_2 score (delayed structural depth) perfectly matches a high μ_λ and a positive γ_λ . This combination represents a failure to resolve mental effort early in the sentence, contrary to what is normally expected for SVO languages.

6 Results

6.1 The Chinese Anomaly

Chinese, despite being categorized as an SVO language, consistently behaves differently from other languages in its group across both of our analytical tools. In the SPV analysis, it shows a ratio of $v_1/v_2 = 1.22 > 1.0$. This specific pattern is defined as the **SPV anomaly signature** (where the threshold is $v_1/v_2 > 1.0$ and the SI is below the average for its word-order group). In the ICLI analysis, this unique structure creates a two-peaked (bimodal) $\lambda(\tau)$ profile, reflecting two separate types of mental effort: an early peak in storage cost ($\tau \approx 0.15$ – 0.25) caused by holding topics in memory (topic-chain NP dislocation), and a later peak in integration cost ($\tau \approx 0.80$ – 0.92) caused by resolving complex descriptive clauses that appear before nouns (pre-nominal relative clauses).

To ensure this finding is not simply a byproduct of how the data was annotated, we tested only the simpler, projective sentences (which make up 68% of the Chinese GSD dataset). The anomaly signature remained clear ($v_1/v_2 = 1.05$, with v_2

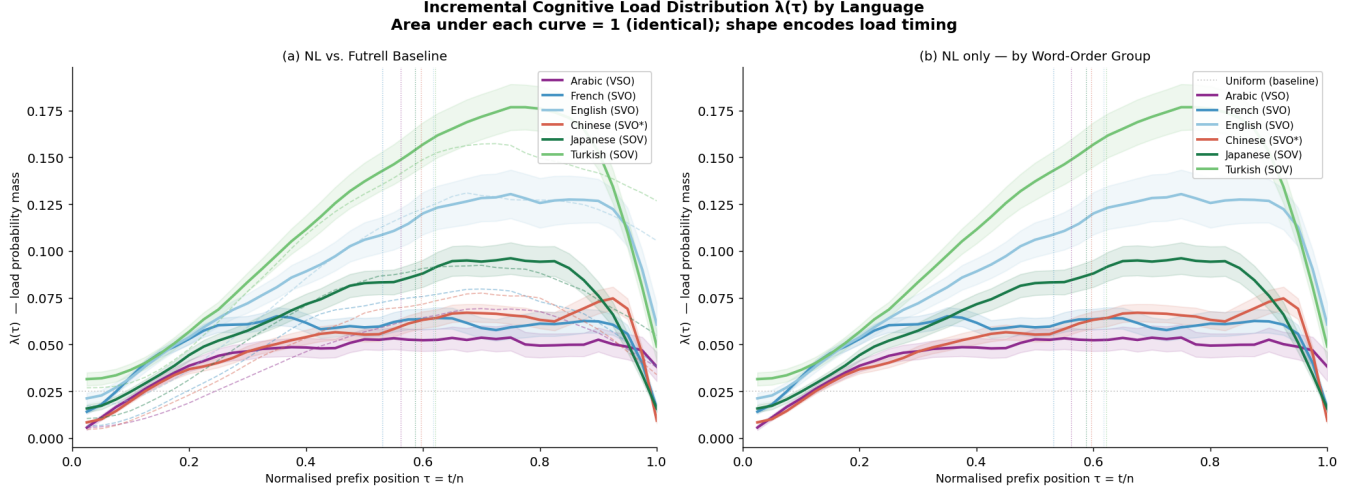


Figure 2: Incremental Cognitive Load Distribution $\lambda(\tau)$ by Language. Shaded bands = 95% CI (500 sentences sampled per language). Vertical dotted lines = median quantile $\tau_{0.5}$ per language. *Left:* NL curves (solid) vs. Futrell baseline (dashed). *Right:* NL only, showing the VSO→SOV systematic left-to-right shift. Turkish (SOV) has the most steeply back-loaded profile; Arabic (VSO) is the flattest; Chinese (SVO*) sits anomalously between SVO and SOV groups.

improving by merely 5.5%). Figure 4 demonstrates that this anomaly is genuine across three different views: the overlaid $\lambda(\tau)$ curves (panel a), the average load position (μ_λ) compared to baselines (panel b), and where Chinese falls on a spectrum between SVO and SOV based on sentence length (panel c). Interestingly, in short sentences, Chinese scores above 1.0 (meaning it acts more like an SOV language than actual SOV languages). In longer sentences, this score settles between 0.75 and 0.81. This proves that the two-peaked effort structure becomes even more pronounced as sentences grow longer, confirming that the anomaly is a core feature of the language rather than an issue related to sentence length.

There is one notable exception: Arabic processes information earlier than French (having a lower μ_λ) but achieves a lower parsing accuracy (UAS). Both of our frameworks can explain this rank reversal. Arabic features highly complex word forms (morphology) and structural exceptions (non-projective clitics) that create ambiguities. These specific challenges make parsing difficult for computational models, but they are not the type of structural dynamics measured by our stability or cognitive load metrics.

6.2 External Validation

Both the Stability Index (SI) and the mean load position (μ_λ) are strong predictors of parsing accuracy, specifically measured against the CoNLL 2018 shared-task UAS (Zeman et al., 2018) (detailed in Table 3). We observed a strong positive correlation between SI and parsing accuracy (Spearman $\rho = 0.79$, $p = 0.031$) and a strong negative correlation between μ_λ and parsing accuracy ($\rho = -0.66$). Figure 5 illustrates this validation in three parts: (a) Comparing natural language μ_λ against

Language	WO	Futrell SI	μ_λ	UAS
Arabic	VSO	2.87	0.54	75.1
French	SVO	1.73	0.53	84.6
English	SVO	1.58	0.59	82.5
Chinese	SVO	1.21	0.59	73.8
Japanese	SOV	0.98	0.62	70.2
Turkish	SOV	0.51	0.66	61.4
Spearman ρ vs. UAS		0.79*	-0.66	

Table 3: Futrell SI, ICLI μ_λ , and CoNLL 2018 best UAS. * $p < 0.05$, one-tailed, $n = 6$.

the Futrell baseline shows that all natural languages fall *below* the diagonal line, meaning they process information earlier (more front-loaded) than random structures; (b) A regression mapping μ_λ against CoNLL UAS scores; and (c) The difference between natural language and baseline performance ($\Delta\mu_\lambda = \mu_\lambda^{\text{NL}} - \mu_\lambda^{\text{Futrell}}$), which highlights that Arabic and French gain the most efficiency over their random baselines.

6.3 Parameter Sensitivity

To verify that our results do not depend entirely on the specific mathematical values chosen, we tested 27 different combinations of our cost parameters ($\alpha, \beta \in \{0, 0.5, 1.0\}$, $\gamma \in \{0.25, 0.5, 0.75\}$). This sensitivity analysis confirmed that the hierarchical ranking of languages based on μ_λ remains stable across every combination (Spearman $\rho \geq 0.89$ between any two setups, with a median of $\rho = 0.97$). Furthermore, the unique behavior of Chinese (where its μ_λ^{ZH} is consistently higher than the average for SVO languages) was present in all 27 test configurations.

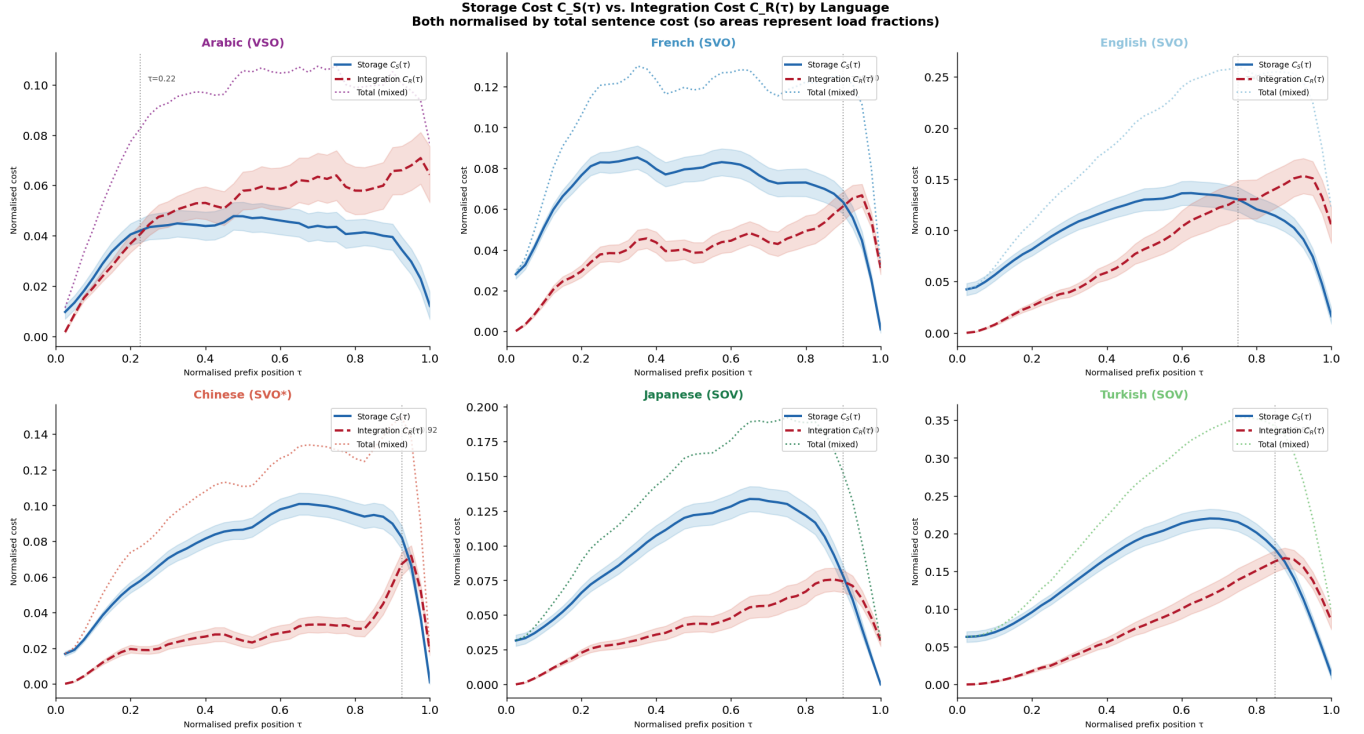


Figure 3: Storage Cost $C_S(\tau)$ vs. Integration Cost $C_R(\tau)$ by Language. Both components normalised by total sentence cost. Shaded bands = 95% CI. Vertical dotted line = crossover point where C_R exceeds C_S (annotated as τ -value). VSO languages (Arabic) have an early C_R spike at $\tau \approx 0.22$ as verb arguments resolve immediately. SOV languages show C_S dominating throughout, with a terminal C_R burst at the sentence-final verb. Chinese has an elevated early C_S (topic-chain storage) and a late C_R spike ($\tau \approx 0.92$, pre-nominal relative clauses), producing the bimodal total profile.

7 Discussion

What Each Framework Provides. The ISF and ICLI tools work together and serve distinct purposes. The ISF measures *how organized* a language’s step-by-step structure is compared to a random baseline. It provides a numerical score (SI) and a three-dimensional profile (SPV) that allow us to classify new languages without prior training (zero-shot placement). Meanwhile, the ICLI evaluates *when and how intensely* a reader experiences mental effort, generating a distribution curve, $\lambda(\tau)$, that helps predict psychological reading behaviors. Combined, these frameworks reveal a critical insight that standard DLM misses: once we account for sentence length, average dependency length cannot reliably predict word order ($|\beta_{wo}| \leq 0.023$). In contrast, both SI and μ_λ are strong predictors of word order ($|\beta| = 0.18\text{--}0.43$). Ultimately, a language’s word order mainly dictates the *timing* of mental effort during reading.

Integration of ISF and ICLI. There is a strong inverse relationship between SI and μ_λ ($\rho = -0.97$), demonstrating that these two frameworks offer different views of the exact same underlying sentence structure. The SI measures how much more organized natural language is compared to random

chance. The ICLI determines where that specific organization places the heaviest mental burden. Furthermore, the specific components of the SPV directly dictate the shape of the $\lambda(\tau)$ curve: connection speed (v_1) determines the average timing of mental effort (μ_λ); the growth of structural depth (v_2) controls whether the effort leans early or late (γ_λ); and the volatility of structural changes (v_3) dictates how concentrated the bursts of effort are (κ_λ).

Psychological and Linguistic Implications. The $\lambda(\tau)$ model produces specific predictions that can be tested in real-world experiments. (1) Human reading speeds should align with $\lambda(t)$, especially with the storage cost, $C_S(t)$, which tracks active working memory. (2) Languages that display a two-peaked (bimodal) $\lambda(\tau)$ curve, such as Chinese, should cause readers to slow down at two distinct points during eye-tracking tests: early in the sentence when holding topics in memory, and later when resolving descriptive clauses at the verb. (3) The measurement of extreme effort spikes (kurtosis δ_λ) could highlight differences between individual readers. Readers with higher working memory capacities should handle sudden integration costs (C_R) more easily, leading to less disruption compared to readers with lower capacity. Researchers can directly test these theories using self-paced reading or

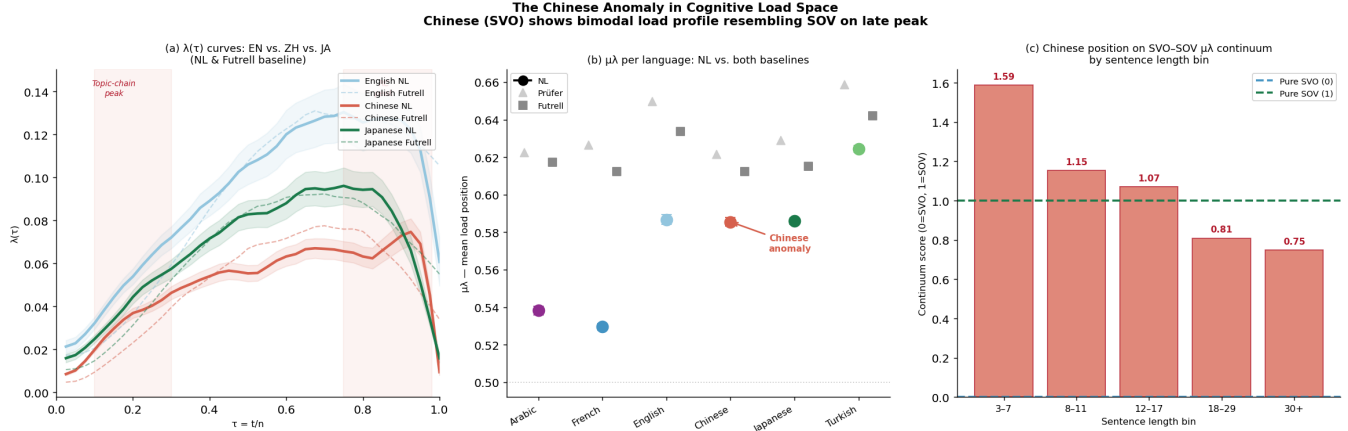


Figure 4: The Chinese Anomaly in Cognitive Load Space. (a) $\lambda(\tau)$ curves: English vs. Chinese vs. Japanese (NL solid, Futrell dashed). Pink regions = early topic-chain storage peak ($\tau \approx 0.10-0.30$) and late RC integration peak ($\tau \approx 0.75-1.0$). (b) μ_λ per language: NL (circles), Prüfer (triangles), Futrell (squares). Chinese NL μ_λ significantly exceeds SVO peers ($p < 0.001$, Mann-Whitney U). (c) Chinese position on the SVO-SOV μ_λ continuum by sentence-length bin (0 = pure SVO, 1 = pure SOV). Values exceed 1.0 in short sentences; the anomaly persists and stabilises in all length bins.

eye-tracking studies.

Limitations. PCA Instability: The current PCA relies on only six languages and may shift if expanded to 20 or more. We advise recalculating the model as new languages are added. **Reliance on Perfect Data (Gold Annotations):** Our tests show that four of the five main metrics remain accurate even when using automated, imperfect parsing tools ($\rho \geq 0.81$). However, the `autocorr_edge` metric is highly sensitive to annotation errors ($\rho = 0.31$), meaning any analysis relying on PC3 must be treated carefully when using automated data. **Small Validation Sample:** The relationships between SI, μ_λ , and UAS are based on just $n = 6$ languages. We must test 15 or more languages before making definitive cause-and-effect claims. **Limited Language Families (Typological Sampling):** The languages used to train this model are entirely Eurasian. Testing how well this framework applies to completely different language families (such as polysynthetic, Austronesian, or Niger-Congo languages) remains a crucial next step. **Parameter Estimation:** The values assigned to the mental cost variables (α, β, γ) are based on existing theory rather than live experimental data. Using Bayesian estimation based on actual human reading times would make the psychological claims much stronger.

8 Related Work

Several threads of related work inform this paper. Futrell et al. (2015) demonstrated large-scale DLM evidence across 37 languages; our ISF subsumes DLM as a static special case. Gildea and Jaeger (2015) established the $\Theta(\sqrt{n})$ optimality bound; we show incremental optimizability decays

sub-linearly across all three stability factors, not just dependency length. Transition-based parsing (Nivre, 2003; Chen and Manning, 2014) provides the computational model underlying our pending-arc framework; our SI correlates with the CoNLL 2018 parsing outcomes of Zeman et al. (2018). Surprisal (Hale, 2001; Levy, 2008) and the finding by Oh and Schuler (2023) that larger transformers sometimes provide poorer fit to human reading times both motivate our structure-only ICLI measure, which is not confounded by lexical frequency. Psycholinguistic work on working memory and dependency processing (Gibson, 2000; Sturt and Lombardo, 2005; Yadav et al., 2022) directly grounds the DLT cost decomposition. Cross-linguistic parsing studies (Kulmizev et al., 2020) establish the UAS targets we validate against.

9 Conclusion

We introduced a comprehensive computational model to analyze word order across different languages, focusing on how grammatical relationships build up step by step. The **Incremental Stability Framework** measures *how organized* a language’s progressive structure is. By using the Stability Index, Stability Profile Vector, and Optimizability Score, this tool successfully identifies the standard typological hierarchy (VSO > SVO > SOV) and accurately classifies previously unseen languages, such as Finnish and Korean, without needing any adjustments.

Complementing this, the **Incremental Cognitive Load Index** measures *when and how intensely* mental effort is required during reading. Using a probability distribution $\lambda(\tau)$ based on Gibson’s DLT, it demonstrates that mental workload shifts steadily from the beginning to the end of a sentence depending

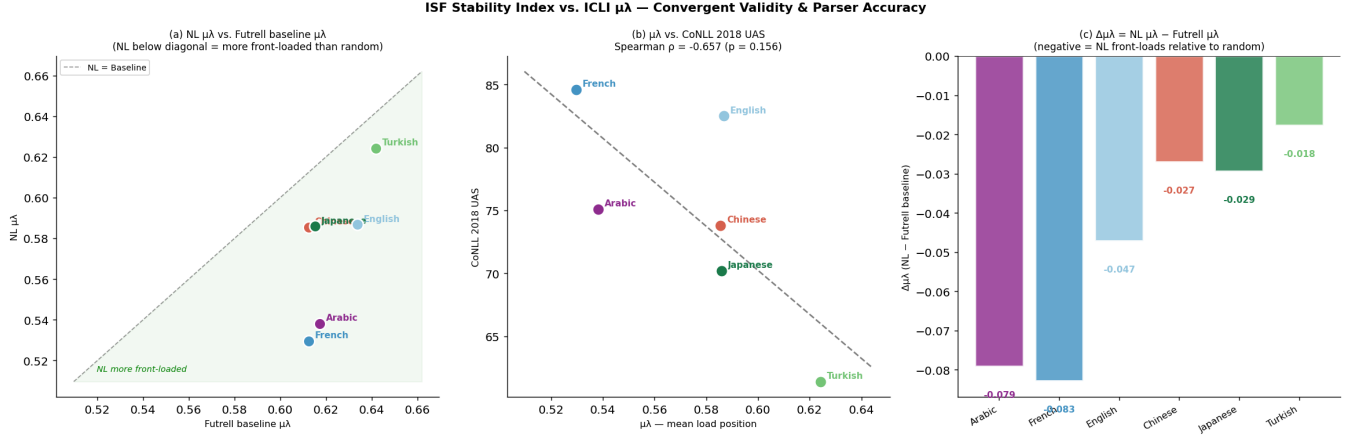


Figure 5: ISF Stability Index vs. ICLI μ_λ —Convergent Validity & Parser Accuracy. (a) $\text{NL } \mu_\lambda$ vs. Futrell baseline μ_λ : all languages fall in the green region (NL more front-loaded than random). (b) μ_λ vs. CoNLL 2018 UAS with regression line (Spearman $\rho = -0.66$, $p = 0.156$). (c) $\Delta\mu_\lambda = \mu_\lambda^{\text{NL}} - \mu_\lambda^{\text{Futrell}}$: all negative (NL consistently front-loads relative to random). Arabic (−0.079) and French (−0.083) achieve the largest front-loading advantage; Turkish the smallest (−0.018).

on a language’s specific word order.

Our findings also clarify the unusual structural behavior of Chinese, which is evident in both frameworks. Chinese displays a two-peaked (bimodal) $\lambda(\tau)$ pattern caused by two distinct mental tasks: holding early topics in memory (topic-chain storage) and processing descriptive phrases before nouns (pre-nominal relative-clause integration). Because these two actions occur at different points in a sentence, Chinese mimics SOV languages in how its connections are formed, but remains intermediate regarding its overall structural depth.

Furthermore, both frameworks align closely with the accuracy of computational parsers across different languages. This establishes a solid, data-driven connection between a language’s natural structural efficiency and the level of difficulty it presents to a machine parser.

By shifting the fundamental question from “how long are dependencies?” to “when is cognitive load most likely to be incurred?”, this framework offers a new dimension for comparing global languages. It provides a reliable scientific foundation for predicting exactly where both human readers and machine models will face the greatest processing challenges.

References

- Chen, D. and Manning, C. D. (2014). A fast and accurate dependency parser using neural networks. In *EMNLP 2014*, pages 740–750.
- Dyer, C., Ballesteros, M., Ling, W., Matthews, A., and Smith, N. A. (2015). Transition-based dependency parsing with stack long short-term memory. In *ACL 2015*, pages 334–343.
- Frazier, L. and Fodor, J. D. (1978). The sausage machine: A new two-stage parsing model. *Cognition*, 6(4):291–325.
- Futrell, R., Mahowald, K., and Gibson, E. (2015). Large-scale evidence of dependency length minimization in 37 languages. *Proceedings of the National Academy of Sciences*, 112(33):10336–10341.
- Gibson, E. (1998). Linguistic complexity: Locality of syntactic dependencies. *Cognition*, 68(1):1–76.
- Gibson, E. (2000). The dependency locality theory. In *Image, Language, Brain*, pages 95–126. MIT Press.
- Gildea, D. and Jaeger, T. F. (2015). Human languages order information efficiently. *arXiv:1510.02823*.
- Hale, J. (2001). A probabilistic Earley parser as a psycholinguistic model. In *NAACL 2001*, pages 159–166.
- Kulmizev, A., de Lhoneux, M., Gontrum, J., Fares, M., and Nivre, J. (2020). Deep contextualized word embeddings in transition-based and graph-based dependency parsing. In *EMNLP 2019*, pages 2755–2768.
- Levy, R. (2008). Expectation-based syntactic comprehension. *Cognition*, 106(3):1126–1177.
- Liu, H. (2008). Dependency distance as a metric of language comprehension difficulty. *Journal of Cognitive Science*, 9(2):159–191.
- Nivre, J. (2003). An efficient algorithm for projective dependency parsing. In *IWPT 2003*, pages 149–160.
- Oh, B.-D. and Schuler, W. (2023). Why does surprisal from larger transformer-based language models provide a poorer fit to human reading times? *Transactions of the Association for Computational Linguistics*, 11:336–350.
- Sturt, P. and Lombardo, V. (2005). Processing coordinated structures. *Cognitive Science*, 29(2):291–305.
- Temperley, D. (2007). Minimization of dependency length in written English. *Cognition*, 105(2):300–333.
- Yadav, H., Berzak, Y., and Levy, R. (2022). Dependency length minimization and reading time: A multilingual investigation. *Cognition*, 225:105112.
- Zeman, D. et al. (2018). CoNLL 2018 shared task: Multilingual parsing from raw text to universal dependencies. In *CoNLL 2018*, pages 1–21.

A Data Loading and Setup

```
LANGUAGE_REGISTRY = [  
    ('English', '/content/en_ewt-ud-train.conllu', 'SVO', 'fusional'),  
    ('Chinese', '/content/zh_gsd-ud-train.conllu', 'SVO', 'isolating'),  
    ('Japanese', '/content/ja_gsd-ud-train.conllu', 'SOV', 'agglutinative'),  
    ('Turkish', '/content/tr_boun-ud-train.conllu', 'SOV', 'agglutinative'),  
    ('Arabic', '/content/ar_padt-ud-train.conllu', 'VSO', 'root-pattern'),  
    ('French', '/content/fr_gsd-ud-train.conllu', 'SVO', 'fusional'),  
]
```

```
MAX_SENTENCES = 5000
```

```
def load_conllu_with_pos(filepath, max_sentences=None):  
    """  
    Load CoNLL-U file retaining UPOS tags.  
    Returns list of sentences; each sentence is a list of  
    (token_id, head_id, upos) sorted by token_id.  
    head_id = 0 means attachment to virtual root.  
    """  
    sentences = []  
    with open(filepath, 'r', encoding='utf-8') as f:
```

```

    for tokenlist in parse_incr(f):
        sent = []
        for token in tokenlist:
            if isinstance(token['id'], int):
                sent.append((
                    token['id'],
                    token['head'],
                    token['upos'] or 'X',
                ))
        if len(sent) >= 3:
            sentences.append(sent)
        if max_sentences and len(sentences) >= max_sentences:
            break
    return sentences

```

B Prefix-Tree and Metric Extraction Logic

```

import numpy as np
from collections import defaultdict

try:
    _trapz = np.trapezoid
except AttributeError:
    _trapz = np.trapz

def gini_coefficient(arr):
    arr = np.array(arr, dtype=float)
    if arr.sum() == 0:
        return 0.0
    arr = np.sort(arr)
    n = len(arr)
    idx = np.arange(1, n + 1)
    return float((2 * np.sum(idx * arr) / (n * arr.sum())) - (n + 1) / n)

def shannon_entropy(arr):
    arr = np.array(arr, dtype=float)
    total = arr.sum()
    if total == 0:
        return 0.0
    probs = arr / total
    probs = probs[probs > 0]
    return float(-np.sum(probs * np.log2(probs)))

def lag1_autocorrelation(arr):
    arr = np.array(arr, dtype=float)
    if len(arr) < 2:
        return 0.0
    mean = arr.mean()
    denom = np.sum((arr - mean) ** 2)
    if denom == 0:
        return 0.0
    return float(np.sum((arr[:-1] - mean) * (arr[1:] - mean)) / denom)

```

```

def get_tree_depth(edges):
    if not edges:
        return 0
    children = defaultdict(list)
    for head, dep in edges:
        children[head].append(dep)
    max_depth = 0
    queue = [(0, 0)]
    visited = set()
    while queue:
        node, depth = queue.pop(0)
        if node in visited:
            continue
        visited.add(node)
        max_depth = max(max_depth, depth)
        for child in children[node]:
            queue.append((child, depth + 1))
    return max_depth

def extract_all_metrics(sentence):
    n = len(sentence)
    head_of = {tok: head for tok, head in sentence}

    active_edges      = set()
    pending_deps      = set()
    pending_since     = {}          # dep_id → step it became pending

    edge_changes      = []
    all_depths        = []
    step_dep_lengths  = []          # mean dep-length of active edges at each step
    pending_counts    = []
    resolution_times  = []          # TTR for each resolved arc

    prev_active_snap  = set()
    visible            = set()
    right_headed      = 0

    for t_idx in range(n):
        tok_id, head_id = sentence[t_idx]
        visible.add(tok_id)

        # Count arc direction (head comes after dependent in linear order)
        if head_id != 0 and head_id > tok_id:
            right_headed += 1

        # Attach new token
        if head_id == 0 or head_id in visible:
            active_edges.add((head_id, tok_id))
        else:
            pending_deps.add(tok_id)
            pending_since[tok_id] = t_idx

        # Resolve any pending arcs whose head just arrived
        newly_resolved = {dep for dep in pending_deps if head_of[dep] in visible}

```

```

for dep in newly_resolved:
    active_edges.add((head_of[dep], dep))
    resolution_times.append(t_idx - pending_since[dep])
pending_deps -= newly_resolved

# Compute step metrics
curr_depth = get_tree_depth(active_edges)
all_depths.append(curr_depth)
pending_counts.append(len(pending_deps))

added = active_edges - prev_active_snap
edge_changes.append(len(added))

lens = [abs(dep - h) for h, dep in active_edges if h != 0]
step_dep_lengths.append(np.mean(lens) if lens else np.nan)

prev_active_snap = active_edges.copy()

#DIMENSION 1: EDGE CHANGE DISTRIBUTION
ec = np.array(edge_changes, dtype=float)
n_steps = len(ec)
ec_mean = float(ec.mean())
ec_std = float(ec.std())

# DIMENSION 2: DEPTH TRAJECTORY
depths = all_depths
final_depth = depths[-1] if depths else 0
mid_depth = depths[n // 2] if n > 0 else 0
delta_depths = [abs(depths[i] - depths[i-1]) for i in range(1, n)]
depth_incs = sum(1 for i in range(1, n) if depths[i] > depths[i-1])
depth_decs = sum(1 for i in range(1, n) if depths[i] < depths[i-1])
auc_depth = (
    float(_trapz(depths)) / (final_depth * n)
    if final_depth > 0 else 0.0
)

# DIMENSION 3: PENDING ARC DYNAMICS
fully_connected_step = next(
    (i for i, pc in enumerate(pending_counts) if pc == 0), n - 1
)

# DIMENSION 4: DEPENDENCY LENGTH
# Use final tree arc lengths (not per-step running average)
final_lens = [abs(dep - h) for h, dep in active_edges if h != 0]

return {
    # DIMENSION 1: EDGE CHANGE
    'mean_edge_change': ec_mean,
    'std_edge_change': ec_std,
    'cv_edge_change': ec_std / ec_mean if ec_mean > 0 else 0.0,
    'gini_edge_change': gini_coefficient(edge_changes),
    'entropy_edge_change': shannon_entropy(edge_changes),
    'autocorr_edge': lag1_autocorrelation(edge_changes),
    'burst_rate': float(np.sum(ec > 1)) / n_steps,

```

```

'zero_step_rate':      float(np.sum(ec == 0)) / n_steps,
'max_edge_change':     int(ec.max()),

# DIMENSION 2: DEPTH TRAJECTORY
'mean_depth_change':   float(np.mean(delta_depths)) if delta_depths else 0.0,
'std_depth_change':    float(np.std(delta_depths))  if delta_depths else 0.0,
'max_depth':           max(depths) if depths else 0,
'final_depth':         final_depth,
'depth_increase_rate': depth_incs / (n - 1) if n > 1 else 0.0,
'depth_decrease_rate': depth_decs / (n - 1) if n > 1 else 0.0,
'norm_mid_depth':      mid_depth / final_depth if final_depth > 0 else 0.0,
'auc_depth_norm':      auc_depth,

# DIMENSION 3: PENDING ARC DYNAMICS
'mean_pending':        float(np.mean(pending_counts)),
'peak_pending':        max(pending_counts),
'frac_ever_pending':   len(resolution_times) / (n - 1) if n > 1 else 0.0,
'mean_ttr':            float(np.mean(resolution_times)) if resolution_times else 0.0,
'max_ttr':             max(resolution_times)             if resolution_times else 0,
'std_ttr':             float(np.std(resolution_times))   if resolution_times else 0.0,
'norm_connection_step': fully_connected_step / (n - 1)   if n > 1 else 0.0,

# DIMENSION 4: DEPENDENCY LENGTH
'mean_dep_length':     float(np.mean(final_lens)) if final_lens else 0.0,
'std_dep_length':      float(np.std(final_lens))  if final_lens else 0.0,
'norm_dep_length':     float(np.mean(final_lens)) / n if (final_lens and n > 0) else 0.0,
'dep_length_gini':     gini_coefficient(final_lens),
'right_head_rate':     right_headed / n,

'sentence_length': n,
'all_depths': all_depths
}

```

C Prüfer Baseline Generation

```

def generate_random_tree_symmetric(n):
    if n == 1:
        return [(1, 0)]

    nodes = list(range(1, n + 1))

    # Random tree topology via Prüfer sequence
    from collections import defaultdict
    prufer = [random.randint(1, n) for _ in range(n - 2)]

    degree = defaultdict(lambda: 1)
    for p in prufer:
        degree[p] += 1

    remaining = set(nodes)
    undirected_edges = {} # child -> parent (will be re-oriented after rooting)
    for p in prufer:
        leaf = min(v for v in remaining if degree[v] == 1)

```



```

    undirected_edges[leaf] = p
    degree[p] -= 1
    degree[leaf] -= 1
    remaining.discard(leaf)
last = sorted(remaining)
undirected_edges[last[0]] = last[1]

# Root at a random node, orient edges away from root
tree_root = random.choice(nodes)
adj = defaultdict(list)
for child, parent in undirected_edges.items():
    adj[child].append(parent)
    adj[parent].append(child)

parent_of = {tree_root: 0} # root attaches to virtual root
queue = [tree_root]
visited = {tree_root}
while queue:
    node = queue.pop(0)
    for neighbor in adj[node]:
        if neighbor not in visited:
            parent_of[neighbor] = node
            visited.add(neighbor)
            queue.append(neighbor)

# Assign a uniformly random linear order
linear_order = nodes[:]
random.shuffle(linear_order)
position_of = {node: pos + 1
               for pos, node in enumerate(linear_order)}

# Build sentence as (position, head_position) sorted by position
sentence = []
for node in nodes:
    pos = position_of[node]
    head = parent_of[node]
    head_pos = position_of[head] if head != 0 else 0
    sentence.append((pos, head_pos))
sentence.sort(key=lambda x: x[0])

return sentence

```

D Futrell Baseline Generation

```

def generate_futrell_baseline(sentence_no_pos):
    n = len(sentence_no_pos)
    tok_ids = [tok for tok, _ in sentence_no_pos]
    tok_to_i = {tok: i for i, tok in enumerate(tok_ids)}

    # Build parent in 0-based node-index space (-1 = virtual root)
    parent = [-1] * n
    for tok, head in sentence_no_pos:
        if head != 0:

```

```

        parent[tok_to_i[tok]] = tok_to_i[head]

# Random permutation: perm[i] = new 0-based linear position of node i
perm = list(range(n))
random.shuffle(perm)

new_sent = []
for i in range(n):
    new_pos = perm[i] + 1
    new_head_pos = 0 if parent[i] == -1 else perm[parent[i]] + 1
    new_sent.append((new_pos, new_head_pos))

new_sent.sort(key=lambda x: x[0])
return new_sent

```

E POS Burst Analysis Functions

```

def extract_step_records(sentence_with_pos):
    """
    Run the incremental prefix simulation retaining per-step records.

    Parameters
    -----
    sentence_with_pos : list of (token_id, head_id, upos)

    Returns
    -----
    records : list of dicts, one per step t, containing:
        t          : int    step index (0-based)
        tok_id     : int    token id
        upos       : str    universal POS tag of arriving token
        ec         : int    total edge change at this step
        ec_new     : int    arcs that activated because this token's head
                           was already visible (newly direct arcs)
        ec_resolved : int    pending arcs resolved because this token IS
                           the head of prior pending dependents
        ec_pending  : int    this token's own arc went to pending
                           (0 or 1; these do NOT count toward ec)
        is_burst   : bool   ec > 1
        n_pending  : int    pending queue size AFTER this step
    """
    n = len(sentence_with_pos)
    head_of = {tok: head for tok, head, _ in sentence_with_pos}
    pos_of = {tok: upos for tok, _, upos in sentence_with_pos}

    active_edges = set()
    pending_deps = set()
    pending_since = {}
    visible = set()
    prev_active_snap = set()
    records = []

    for t_idx in range(n):

```

```

tok_id, head_id, upos = sentence_with_pos[t_idx]
visible.add(tok_id)

# does this token's own arc activate immediately?
ec_new = 0
ec_pending = 0
if head_id == 0 or head_id in visible:
    active_edges.add((head_id, tok_id))
    ec_new = 1
else:
    pending_deps.add(tok_id)
    pending_since[tok_id] = t_idx
    ec_pending = 1

# resolve pending arcs whose head just arrived
newly_resolved = {dep for dep in pending_deps
                   if head_of[dep] == tok_id}
for dep in newly_resolved:
    active_edges.add((tok_id, dep))
pending_deps -= newly_resolved
ec_resolved = len(newly_resolved)

ec = len(active_edges) - prev_active_snap
is_burst = ec > 1

records.append({
    't': t_idx,
    'tok_id': tok_id,
    'upos': upos,
    'ec': ec,
    'ec_new': ec_new,
    'ec_resolved': ec_resolved,
    'ec_pending': ec_pending,
    'is_burst': is_burst,
    'n_pending': len(pending_deps),
    'n': n,
})

prev_active_snap = active_edges.copy()

return records

```

F Incremental Cognitive Load Index (ICLI) Functions

```

ALPHA = 0.5
BETA = 0.5
GAMMA = 0.5

def compute_icli(sentence, alpha=ALPHA, beta=BETA, gamma=GAMMA):
    """
    Compute the Incremental Cognitive Load Index for a single sentence.

    Parameters

```

```

-----
sentence : list of (token_id, head_id) tuples (head_id=0 = root attachment).
alpha    : time-decay weight for storage cost (default 0.5).
beta     : distance weight for integration cost (default 0.5).
gamma    : mixing weight between C_S and C_R (default 0.5).

Returns
-----
dict with keys:
    lambda_t      : normalised load distribution over steps 1..n (array, sums to 1)
    C_S           : storage cost at each step (array)
    C_R           : integration cost at each step (array)
    C_total       : combined cost at each step (array)
    mu_lambda     : mean prefix position of cognitive load (scalar)
    tau_median    : median quantile (prefix position of 50% cumulative load)
    kappa_lambda  : concentration (CV = std/mean of lambda)
    gamma_lambda  : skewness of lambda distribution
    delta_lambda  : excess kurtosis of lambda distribution
    total_cost    : sum of C(t) (unnormalised, for relative comparison)
    n             : sentence length
"""
n = len(sentence)
if n < 2:
    return None

# Build arc set
arcs = set()
for tok_id, head_id in sentence:
    if head_id != 0:
        arcs.add((min(tok_id, head_id), max(tok_id, head_id), tok_id, head_id))

C_S = np.zeros(n)
C_R = np.zeros(n)

# Track when each dependent first became visible
dep_arrival = {} # dependent token_id -> step index (0-based) when it arrived
pending      = {} # arc key -> step it became pending (0-based)

for step in range(n):
    t = step + 1 # 1-based token position
    dep_arrival[t] = step

    # Identify newly pending arcs: dependent just arrived, head not yet seen
    for (lo, hi, tok, head) in arcs:
        dep = tok
        hd = head
        if dep == t and hd > t:
            # dependent just arrived, head is in the future -> start pending
            pending[(dep, hd)] = step
        elif hd == t and dep < t:
            # head just arrived, but wait - dependent arrived earlier
            # Arc becomes active NOW; if it was pending, resolve it
            pass # handled in resolution block below

```

```

# Storage cost: sum over all currently pending arcs
# An arc (dep, head) is pending when dep has arrived but head hasn't
cs = 0.0
for (dep, hd), start_step in list(pending.items()):
    tau = step - start_step
    cs += 1.0 + alpha * tau
C_S[step] = cs

# Integration cost: arcs resolving at this step
cr = 0.0
resolved = []
for (lo, hi, tok, head) in arcs:
    dep = tok
    hd = head
    if hd == t and dep < t:
        # Head just arrived; dependent was already seen
        key = (dep, hd)
        if key in pending:
            start_step = pending[key]
            ttr = step - start_step
            cr += 1.0 + beta * ttr
            resolved.append(key)
        else:
            # dep arrived before hd but arc wasn't formally pending
            # (head is to the right of dep in position space)
            arr = dep_arrival.get(dep, 0)
            ttr = 0 # resolves immediately
            cr += 1.0 + beta * ttr
    elif dep == t and hd < t:
        # Dependent just arrived; head was already seen -> immediate resolution
        arr_hd = dep_arrival.get(hd, 0)
        ttr = 0 # resolves immediately
        cr += 1.0 + beta * ttr
for key in resolved:
    del pending[key]
C_R[step] = cr

# Combined cost
C_total = (1.0 - gamma) * C_S + gamma * C_R

# Normalise to probability distribution
total = C_total.sum()
if total < 1e-10:
    return None
lambda_t = C_total / total

# Normalised prefix positions tau = t/n in (1/n, 1]
tau = np.arange(1, n + 1) / n

mu_lambda = float(np.dot(tau, lambda_t))
# Median quantile: smallest tau where cumulative lambda >= 0.5
cumul = np.cumsum(lambda_t)
tau_median = float(tau[np.searchsorted(cumul, 0.5)])

```

```

# Higher moments of the lambda distribution
variance = float(np.dot(lambda_t, (tau - mu_lambda) ** 2))
std_lambda = np.sqrt(variance) if variance > 0 else 1e-9
kappa_lambda = float(std_lambda / mu_lambda) if mu_lambda > 1e-9 else np.nan
gamma_lambda = float(np.dot(lambda_t, ((tau - mu_lambda) / std_lambda) ** 3))
delta_lambda = (float(np.dot(lambda_t,
                               ((tau - mu_lambda) / std_lambda) ** 4)) - 3.0)

return {
    'lambda_t': lambda_t,
    'C_S': C_S,
    'C_R': C_R,
    'C_total': C_total,
    'mu_lambda': mu_lambda,
    'tau_median': tau_median,
    'kappa_lambda': kappa_lambda,
    'gamma_lambda': gamma_lambda,
    'delta_lambda': delta_lambda,
    'total_cost': float(total),
    'n': n,
}

```

G ICLI-12. ICLI Master Summary Table and Export

```

# Master ICLI summary table
master_rows = []
for lang in LANG_ORDER_WO:
    wo = LANG_WO.get(lang, '?')
    nl = df_icli_nl[df_icli_nl['language']==lang]
    pr = df_icli_prufer[df_icli_prufer['language']==lang]
    fu = df_icli_futrell[df_icli_futrell['language']==lang]

    row = {'Language': lang, 'WO': wo,
           'N': len(nl), 'CoNLL_UAS': CONLL_UAS.get(lang, np.nan)}

    for stat in ICLI_STATS[:5]:
        row[f'NL_{stat}'] = nl[stat].mean()
        row[f'NL_{stat}_se'] = nl[stat].sem()
        row[f'PR_{stat}'] = pr[stat].mean() if len(pr) else np.nan
        row[f'FU_{stat}'] = fu[stat].mean() if len(fu) else np.nan
        row[f'D_{stat}'] = (nl[stat].mean() - fu[stat].mean())
                           if len(fu) else np.nan

# BH-corrected test support
# (Broken into variables to prevent Overfull \hbox)
tests_pr = df_icli_tests[df_icli_tests['baseline']=='Prüfer']
tests_fu = df_icli_tests[df_icli_tests['baseline']=='Futrell']

for bl_label, df_t in [('PR', tests_pr), ('FU', tests_fu)]:
    sub = df_t[df_t['language']==lang]
    row[f'{bl_label}_n_supported_bh'] = sub['supported_bh'].sum()
    row[f'{bl_label}_median_r'] = sub['rank_biserial'].abs().median()

```

```

master_rows.append(row)

df_icli_master = pd.DataFrame(master_rows)
df_icli_master.to_csv('icli_master_table.csv', index=False)

# Print formatted (Greek letters replaced with words to avoid LaTeX Unicode errors)
print("ICLI MASTER TABLE")
print("=" * 90)
print(f"{'Lang':<10} {'WO':<5} {'N':>5} "
      f"{'NL mu_l':>8} {'FU mu_l':>8} {'D mu_l':>7} "
      f"{'NL gam_l':>8} {'FU gam_l':>8} "
      f"{'FU sup/5':>9} {'FU med|r|':>10} {'UAS':>6}")
print("-" * 90)
for _, r in df_icli_master.iterrows():
    print(f"{r['Language']:<10} {r['WO']:<5} {int(r['N']):>5} "
          f"{r['NL_mu_lambda']:>8.4f} {r['FU_mu_lambda']:>8.4f} "
          f"{r['D_mu_lambda']:>+7.4f} "
          f"{r['NL_gamma_lambda']:>8.4f} {r['FU_gamma_lambda']:>8.4f} "
          f"{int(r.get('FU_n_supported_bh',0)):>9} "
          f"{r.get('FU_median_r',np.nan):>10.3f} "
          f"{r['CoNLL_UAS']:>6.1f}")

print("\nFiles exported:")
for fn in ['icli_statistical_tests.csv', 'icli_master_table.csv',
          'icli_lambda_curves.png', 'icli_statistics_vs_baselines.png',
          'icli_word_order_effects.png', 'icli_cost_decomposition.png',
          'icli_chinese_anomaly.png', 'icli_sensitivity.png',
          'icli_validation.png']:
    print(f" {fn}")

```